



Operating Systems

Main Memory-Part2

Seyyed Ahmad Javadi

sajavadi@aut.ac.ir

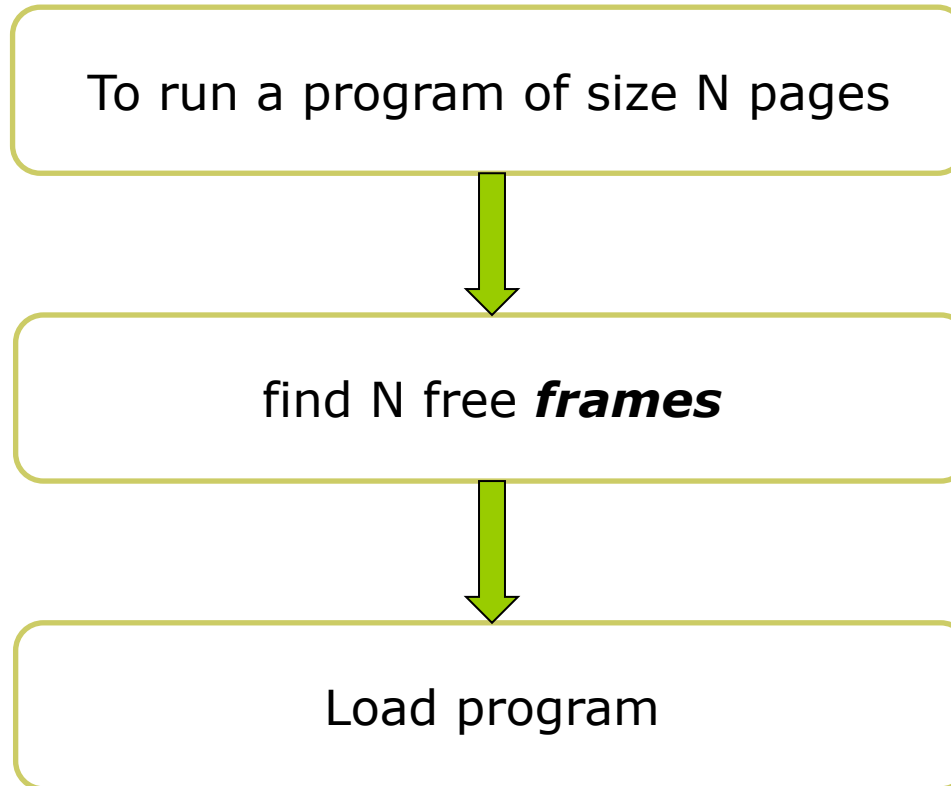
Fall 2025

Paging

- Physical address space of a process can be **noncontiguous**.
- Process is allocated physical memory whenever **the latter** is available
 - **Avoids external fragmentation**
 - Avoids problem of varying sized memory chunks
- Divide physical memory into fixed-sized blocks called **frames**
 - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called **pages**

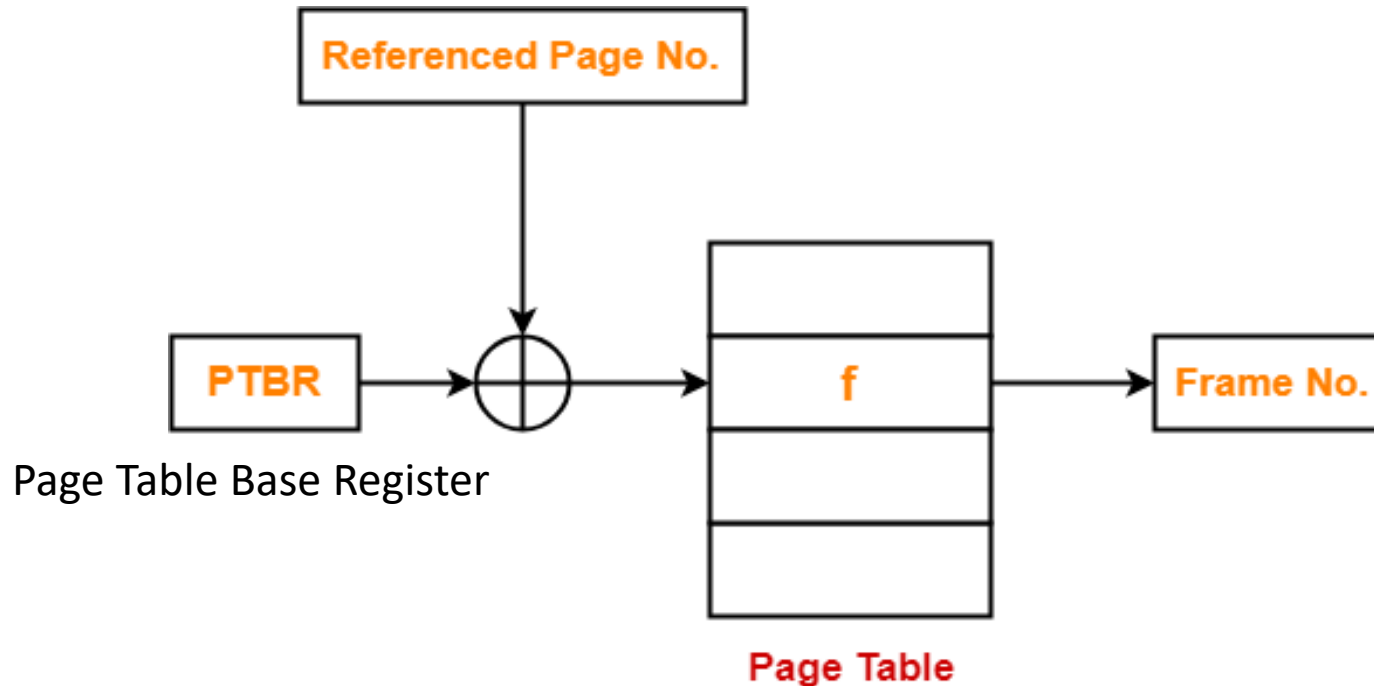
Paging (cont.)

- Keep track of **all free frames**



Paging (cont.)

- Set up a **page table** to translate logical to physical addresses

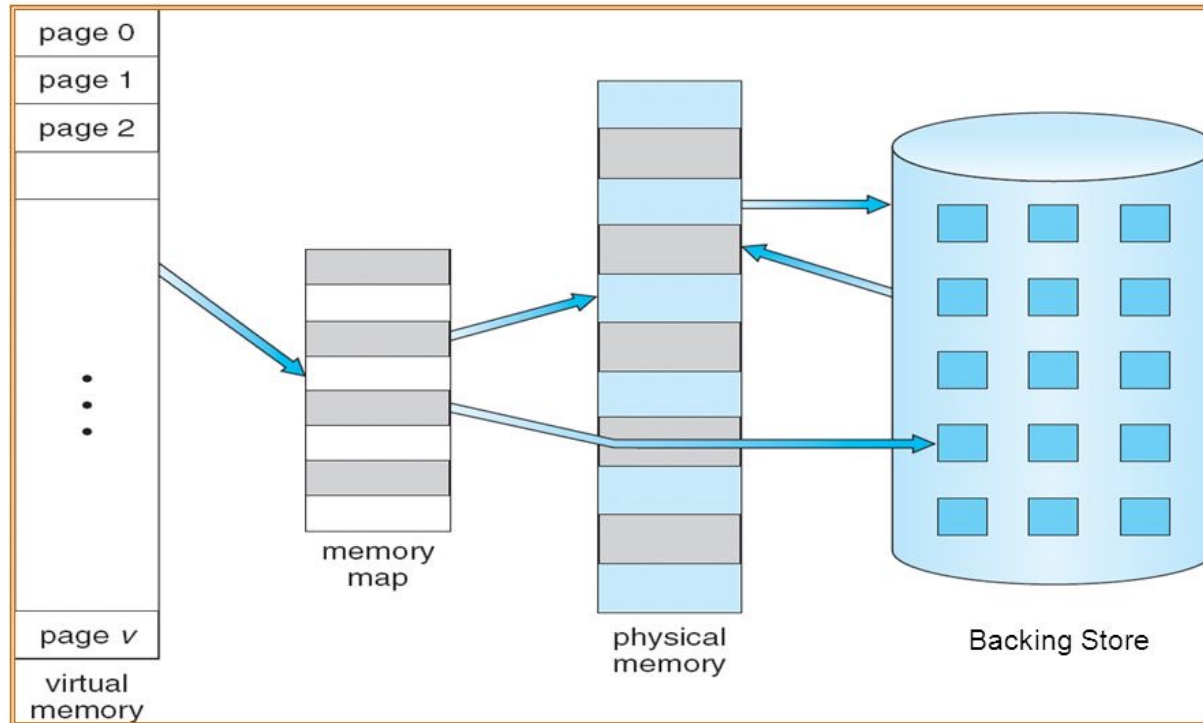


Obtaining Frame Number Using Page Table

Source: <https://www.gatevidyalay.com/page-table-paging-in-operating-system/>

Paging (cont.)

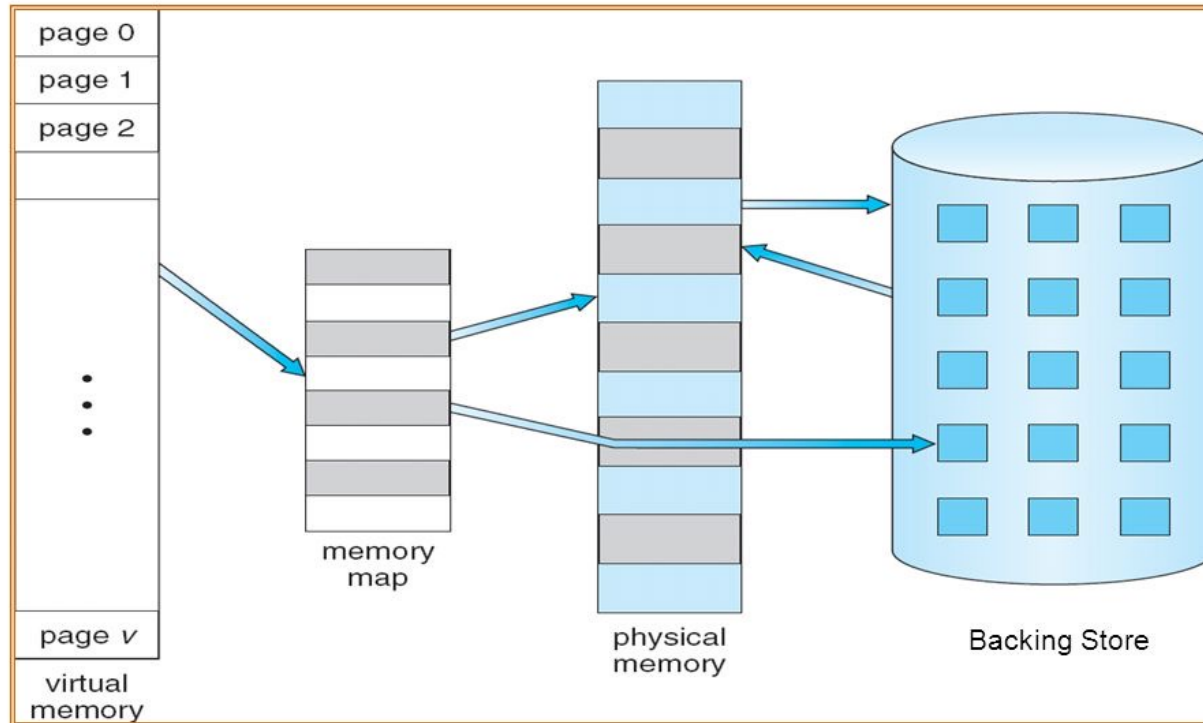
- **Backing store likewise split into pages**



- **Does paging solve fragmentation issue?**

Paging (cont.)

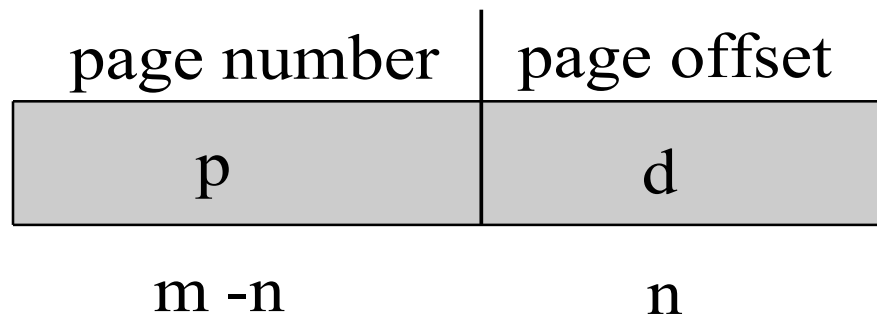
- **Backing store likewise split into pages**



- **Still have Internal fragmentation**

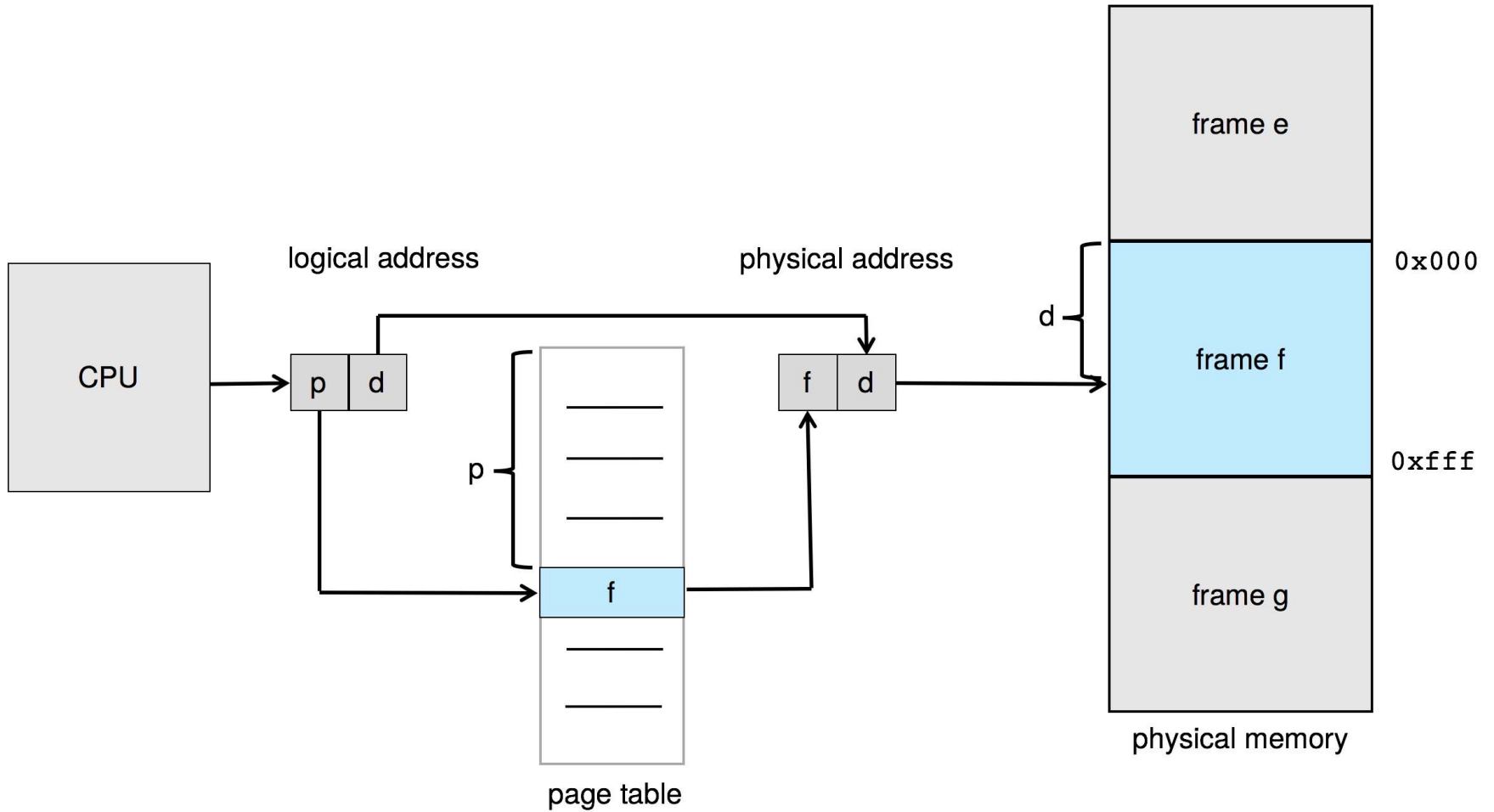
Address Translation Scheme

- Address generated by CPU is divided into:
 - Page number (p) – used as an index into a page table which contains base address of each page in physical memory
 - Page offset (d) – combined with base address to define the physical memory address that is sent to the memory unit

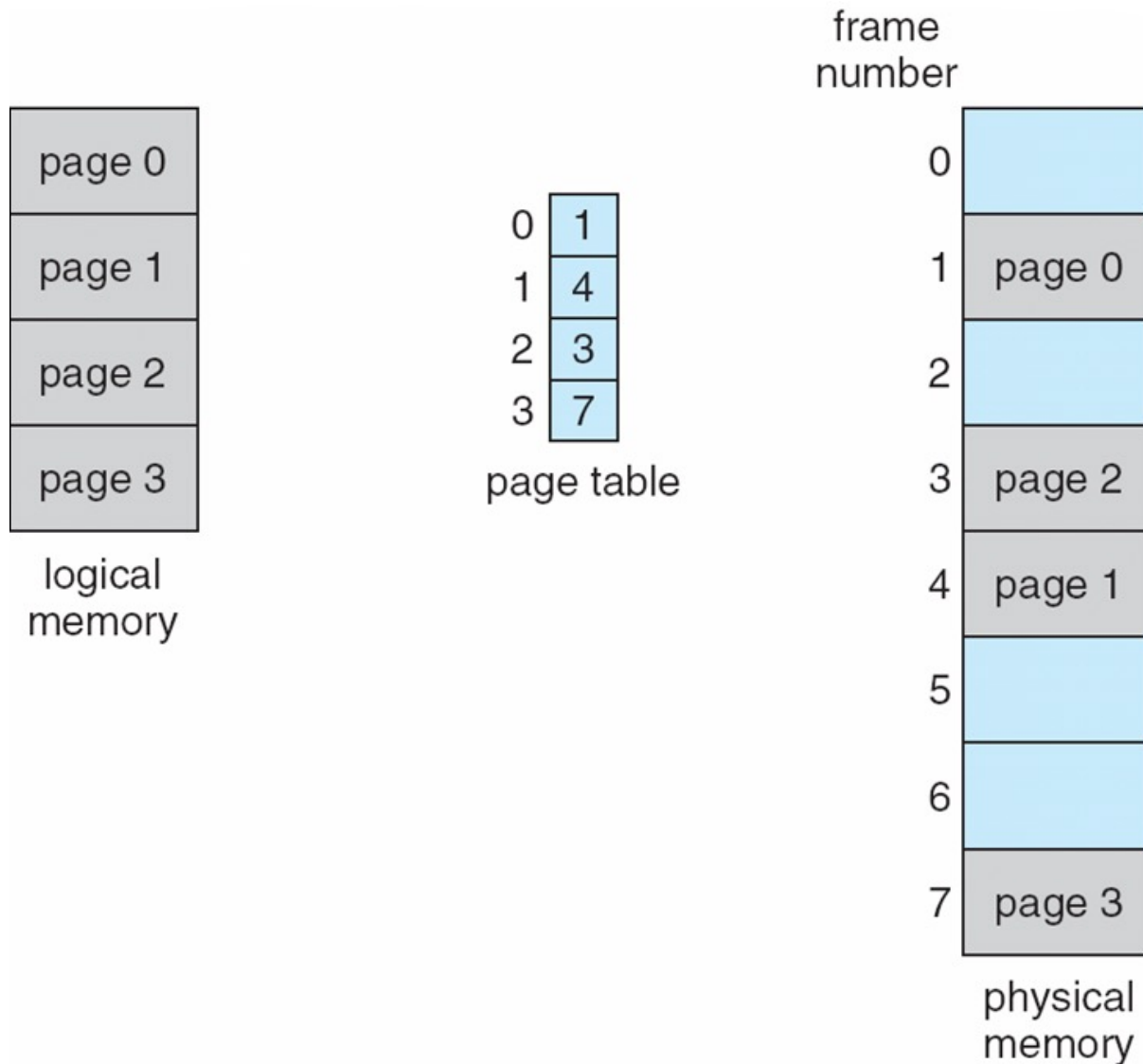


- For given logical address space 2^m and page size 2^n

Paging Hardware



Paging Model of Logical and Physical Memory



Paging Example

- Logical address: $n = 2$ and $m = 4$.

Using a page size of 4 bytes
and a physical memory of
32 bytes (8 pages)

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Paging -- Calculating internal fragmentation

- Page size = 2,048 bytes
- Process size = 72,766 bytes
- 35 pages + 1,086 bytes
- Internal fragmentation of ?
- Worst case fragmentation = ?
- On average fragmentation = ?

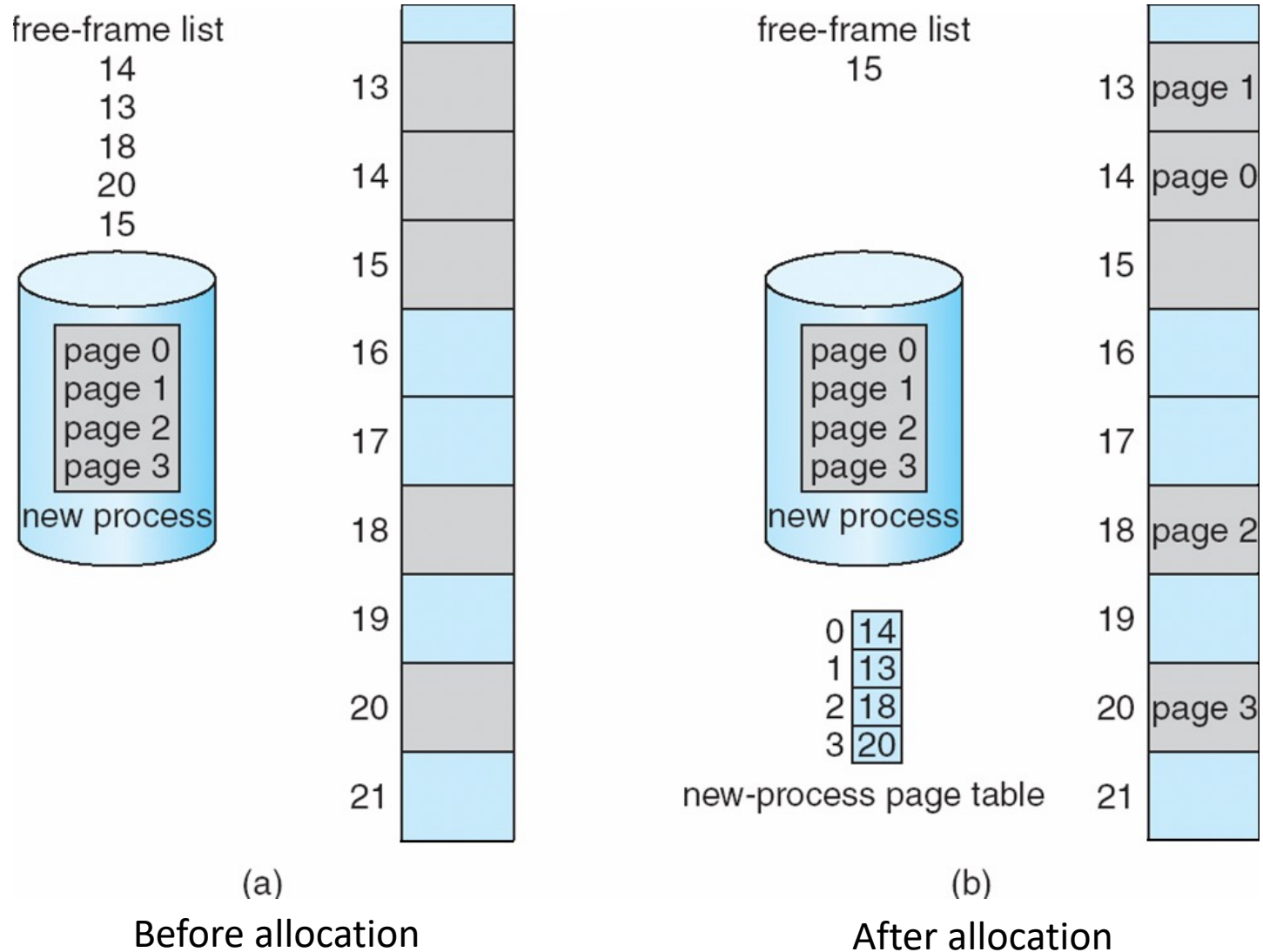


Paging -- Calculating internal fragmentation

- Page size = 2,048 bytes
- Process size = 72,766 bytes
- 35 pages + 1,086 bytes
- Internal fragmentation of $2,048 - 1,086 = 962$ bytes
- Worst case fragmentation = 1 frame – 1 byte
- On average fragmentation = $1 / 2$ frame size
- So small frame sizes desirable?
- But each page table entry takes memory to track
- Page sizes growing over time
 - Solaris supports two page sizes: 8 KB and 4 MB



Free Frames



Implementation of Page Table

- Page table is kept in main memory
 - **Page-table base register (PTBR)** points to the page table
 - **Page-table length register (PTLR)** indicates size of the page table

- In this scheme every data/instruction access requires two memory accesses
 - One for the page table
 - One for the data / instruction



Implementation of Page Table

- The two-memory access problem can be solved by the use of a special fast-lookup hardware cache called **translation look-aside buffers (TLBs)**
 - Also called **associative memory**.
- TLBs typically small (64 to 1,024 entries)



Translation Look-Aside Buffer

- Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry
 - Uniquely identifies each process to provide address-space protection for that process.
 - What if ASIDs is not supported?
 - ▶ Otherwise need to flush at every context switch.



Translation Look-Aside Buffer

- On a TLB miss, value is loaded into the TLB for faster access next time
 - Replacement policies must be considered
 - Some entries can be **wired down** for permanent fast access



Hardware

- Associative memory

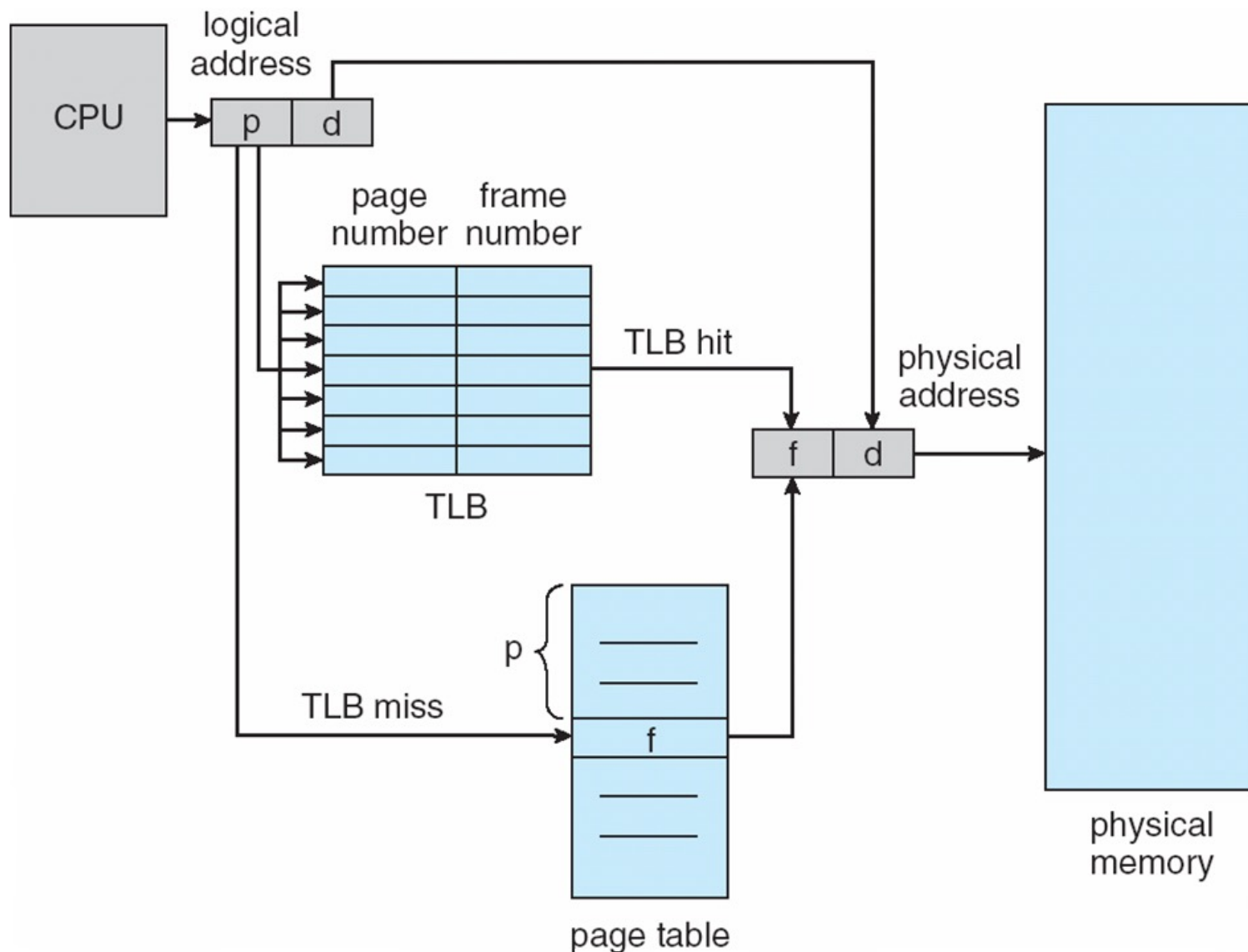
- **Parallel search**

Page #	Frame #

- Address translation (p, d)

- If p is in associative register, get frame # out
 - Otherwise get frame # from page table in memory

Paging Hardware With TLB



Effective Access Time

- Hit ratio: percentage of times that a page number is found in the TLB
- An 80% hit ratio means that we find the desired page number in the TLB 80% of the time.
- Suppose that 10 nanoseconds to access memory.
 - If the desired page in TLB then a mapped-memory access take 10ns.
 - Otherwise, we need two memory access so it is 20 ns



Effective Access Time (cont.)

- **Effective Access Time (EAT)**

$$\text{EAT} = 0.80 \times 10 + 0.20 \times 20 = 12 \text{ nanoseconds}$$

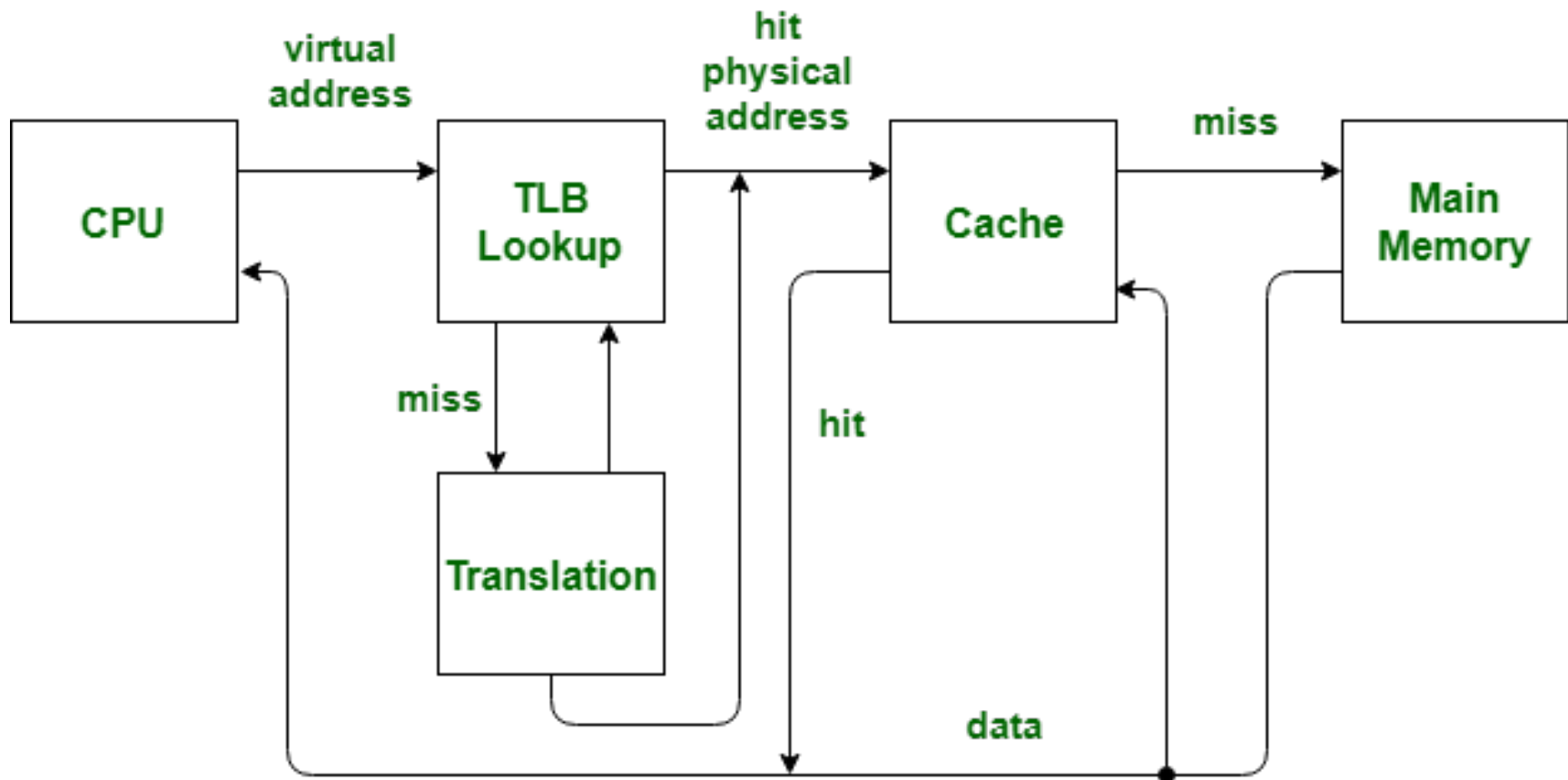
implying 20% slowdown in access time

- Consider a more realistic hit ratio of 99%,

$$\text{EAT} = 0.99 \times 10 + 0.01 \times 20 = 10.1\text{ns}$$

implying only 1% slowdown in access time.

Cache vs TLB



- <https://stackoverflow.com/questions/1973473/difference-between-cache-and-translation-lookaside-buffertlb>
- <https://www.geeksforgeeks.org/whats-difference-between-cpu-cache-and-tlb/>

Memory Protection

- Memory protection implemented by ***associating protection bit*** with each frame to indicate if read-only or read-write access is allowed.
 - Can also add more bits to indicate page execute-only, and so on



Memory Protection

- Valid-invalid bit attached to each entry in the page table:
 - **valid**: the associated page is in the process' logical address space, and is thus a legal page
 - **invalid**: the page is not in the process' logical address space
- Or use page-table length register (PTLR)
- Any violations result in a trap to the kernel



Valid (v) or Invalid (i) Bit In A Page Table

00000	page 0
	page 1
	page 2
	page 3
	page 4
10,468	page 5
12,287	

frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

0	
1	
2	page 0
3	page 1
4	page 2
5	
6	
7	page 3
8	page 4
9	page 5
	⋮
	page n

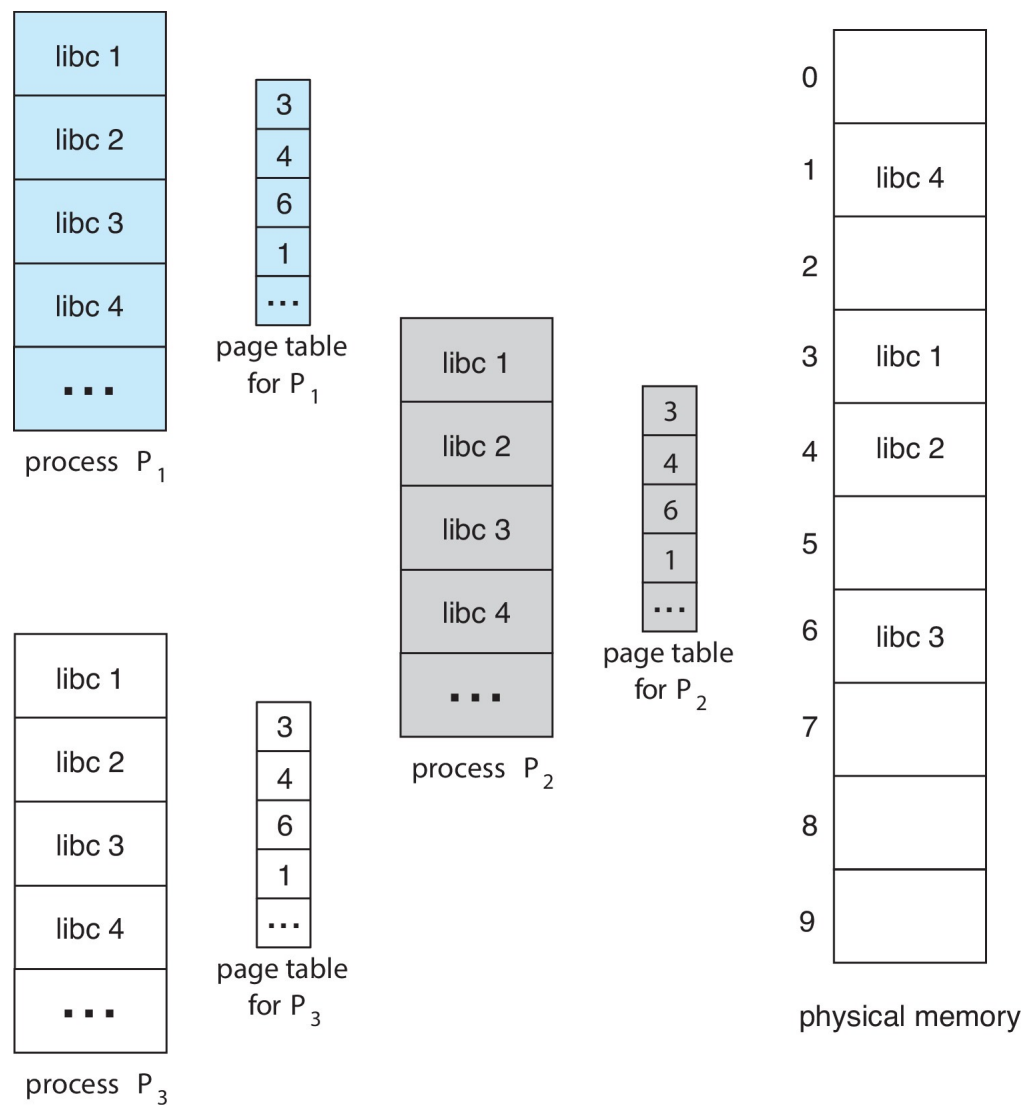
Shared Pages

■ Shared code

- One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)



Shared Pages Example



Shared Pages

■ Shared code

- One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
- Similar to multiple threads sharing the same process space
- Also useful for interprocess communication if sharing of read-write pages is allowed.

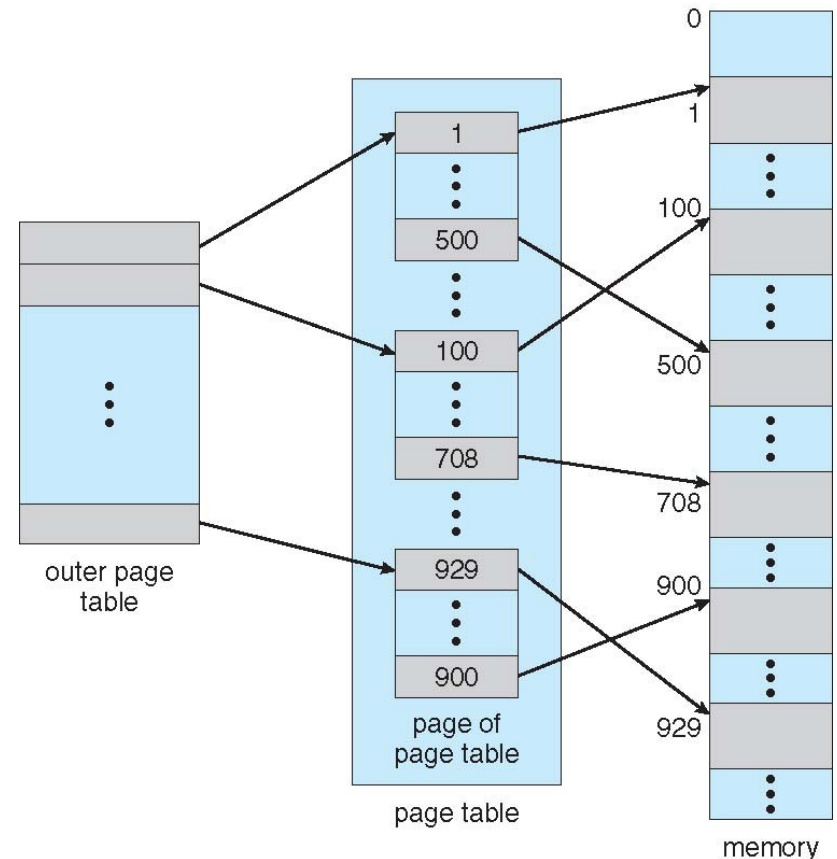
■ Private code and data

- Each process keeps a separate copy of the code and data
- The pages for the private code and data can appear anywhere in the logical address space.



Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table



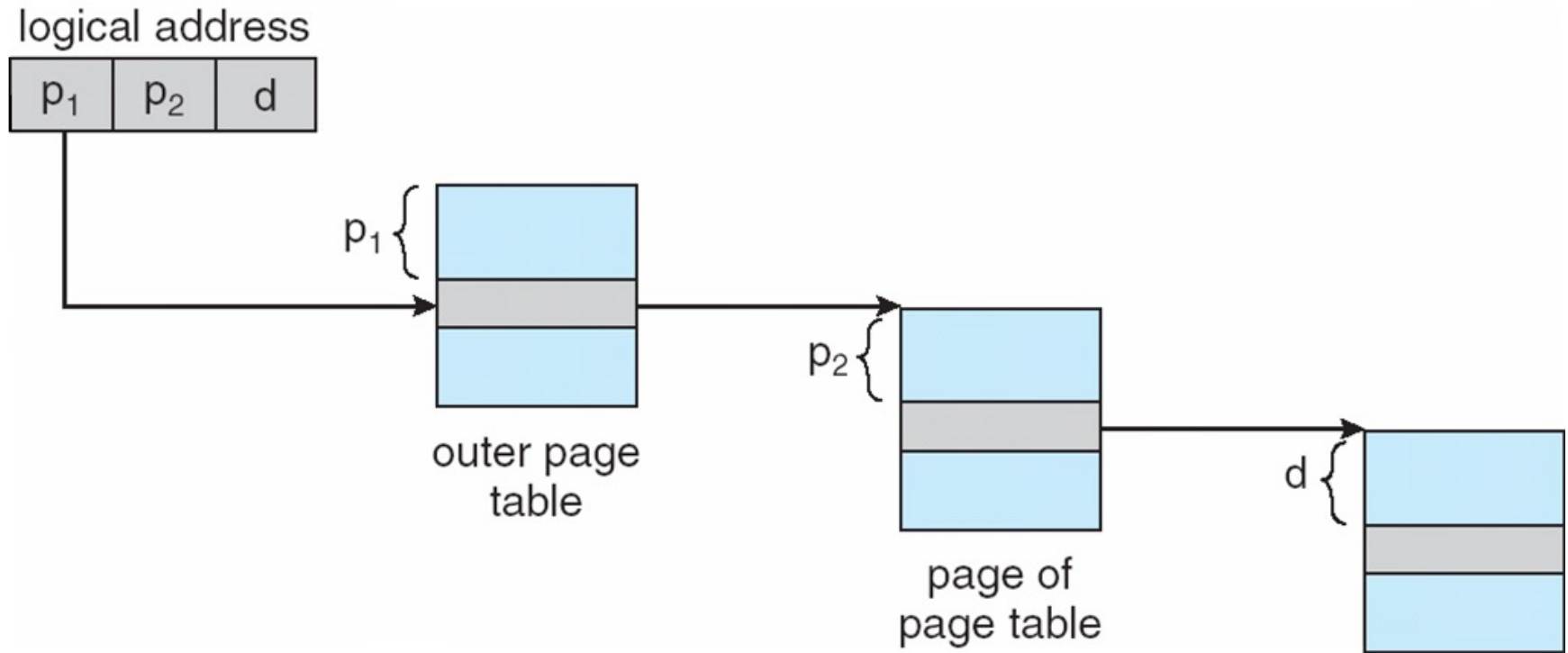
Two-Level Paging Example

- A logical address (on 32-bit machine with 4K page size) is divided into:
 - a page number consisting of 20 bits
 - a page offset consisting of 12 bits
- Since the page table is paged, the page number is further divided into:
 - a 10-bit page number
 - a 10-bit page offset
- Thus, a logical address is as follows:

page number		page offset
p_1	p_2	d
10	10	12
- Where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table
- Known as **forward-mapped page table**



Address-Translation Scheme

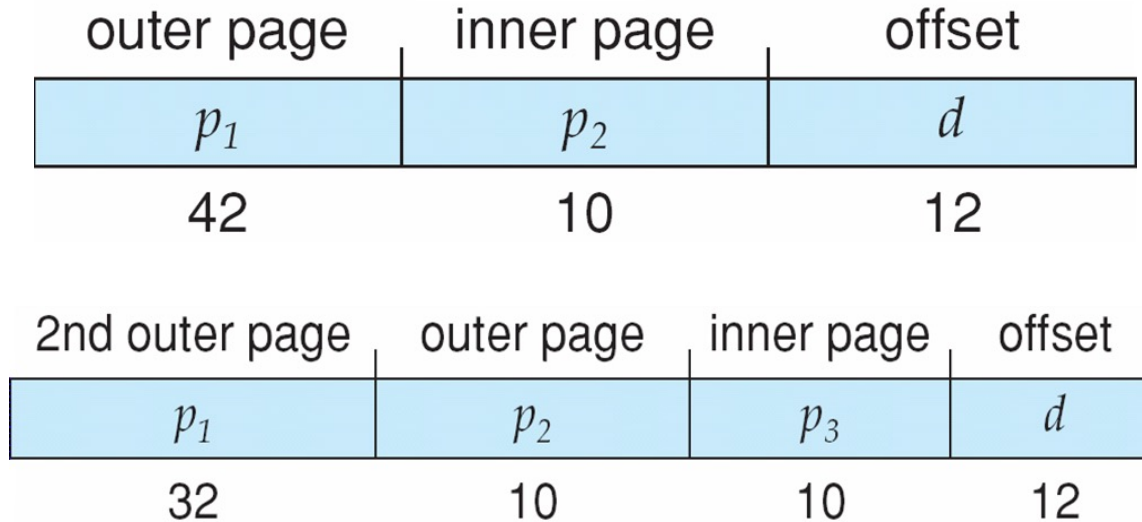


64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
 - Then page table has 2^{52} entries
 - If two level scheme, inner page tables could be 2^{10} 4-byte entries
 - Address would look like

outer page	inner page	offset
p_1	p_2	d
42	10	12
 - Outer page table has 2^{42} entries or 2^{44} bytes
 - One solution is to add a 2nd outer page table
 - But in the following example the 2nd outer page table is still 2^{34} bytes in size
 - ▶ And possibly 4 memory access to get to one physical memory location

Three-level Paging Scheme



In general, is it ***appropriate*** to use hierarchical page tables for 64 bit architecture?

x86-64 example

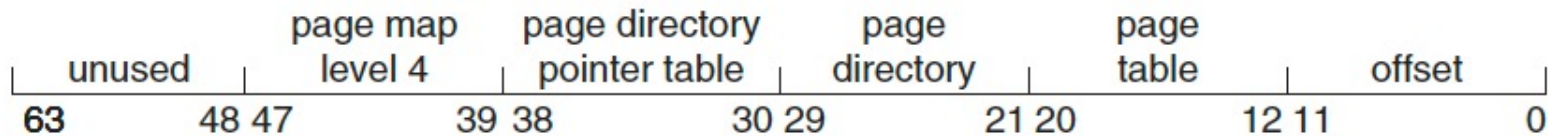


Figure 9.25 x86-64 linear address.

Physical Memory in Bytes [All Values are in Hexidecimal]

Address	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
00000000	0E	0F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D
00000010	1E	1F	20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D
...																
00001010	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
00001020	40	03	41	01	30	01	31	03	00	03	00	00	00	00	00	00
00001030	00	11	22	33	44	55	66	77	88	99	AA	BB	CC	DD	EE	FF
00001040	10	01	11	02	31	03	13	04	14	01	15	03	16	01	17	00
...																
00002030	10	01	11	00	12	03	67	03	11	03	00	00	00	00	00	00
00002040	02	20	03	30	04	40	05	50	01	60	03	70	08	80	09	90
00002050	10	00	31	01	10	03	31	01	12	03	30	00	10	00	10	01
...																
00004000	00	00	11	01	11	01	33	03	34	01	35	00	43	38	32	79
00004010	50	28	84	19	71	69	39	93	75	10	58	20	97	49	44	59
00004020	23	03	20	03	00	01	62	08	99	86	28	03	48	25	34	21
...																
00100000	00	00	10	65	00	00	20	67	00	00	30	00	00	00	40	07
00100010	00	00	50	03	00	00	00	00	00	00	20	67	00	00	00	00
...																
00102000	00	00	20	01	00	00	00	00	00	00	30	03	00	00	40	07
...																
00103000	11	22	00	05	55	66	77	88	99	AA	BB	CC	DD	EE	FF	00
00103010	22	33	44	55	66	77	88	99	AA	BB	CC	DD	EE	FF	00	67
...																
001FE000	04	15	00	00	48	59	70	7B	8C	9D	AE	BF	D0	E1	F2	03
001FE010	10	15	00	67	10	15	10	67	10	15	20	67	10	15	30	67
...																
001FF000	00	00	20	00	00	00	10	65	00	00	10	67	00	00	20	65
001FF010	00	00	20	67	00	00	30	67	00	00	40	65	00	00	50	07
...																
001FFFF0	00	00	00	00	00	00	00	00	10	00	00	67	00	00	40	65
...																
00200000	00	1F	F0	07	00	10	10	07	00	10	20	07	00	10	30	07
00200010	00	10	20	07	00	10	50	07	00	10	60	07	00	10	70	07
00200020	00	10	00	07	00	00	00	00	00	00	00	00	00	00	00	00
...																
00200FF0	00	00	00	00	00	00	00	00	00	1F	E0	07	00	1F	F0	07
...																

PTBR



Address translation example

Virtual address	Virtual page #1	Virtual page #2	First-Level PTE	Next-Level Page-Table Address	Second-Level PTE	Final Physical Address
0x00001047	0x000	0x001	0x001FF007	0x001FF000	0x00001065	0x00001047
0x00C07665						
0xFFFFF555						

Virtual page #1										Virtual page #2										offset											
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	1	1	1
0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	1	1	1	0	1	1	0	0	1	1	0	0	1	0	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1

